

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Харківський національний університет імені В.Н. Каразіна

Навчально-науковий інститут комп'ютерних наук та штучного інтелекту

Кафедра математичного моделювання та аналізу даних

ЛАБОЛАТОРНА РОБОТА № 6

на тему: «Робота з індексами та курсорами в MSSQL»

Виконав: студент 3 курсу групи кс31
Спеціальності 124 «Комп'ютерні
науки»

Кірієнко Данило

Прийняв: к.т.н., доцент каф. ММ та
АД

Коробчинський К.П.

Мета роботи:

Ознайомитися з поняттям індексів у системі керування базами даних Microsoft SQL Server, вивчити їх призначення, особливості використання та вплив на продуктивність запитів. Засвоїти оператори створення індексів, порядок їх застосування, а також методи реорганізації та видалення. Набути практичних навичок роботи з індексами шляхом створення, оптимізації та аналізу їх ефективності у базі даних.

Завдання 2.

За своїм варіантом (основного, по якому були виконані практичні роботи 3, 4, 5 та 6) додати опис предметної області (скопіювати з варіанту), логічну і фізичну моделі (скопіювати з практичної роботи №5). Наступні запити виконати в зручному для Вас середовищі: Docker, Віртуальна машина з MSSQL, файлова БД або встановлений на локальному комп'ютері екземпляр MSSQL.

Завдання 3.

За Вашим варіантом задання реалізуйте транзакцію з двома операціями та умовою, яка призведе до ROLLBACK.

Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл CUSOR.SQL та оновіть його на своїй сторінці на GitHub.

Завдання 4.

За Вашим варіантом задання виконайте перевірку @@ERROR для керування транзакцією.

Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл CUSOR.SQL та оновіть його на своїй сторінці на GitHub.

Завдання 5.

За Вашим варіантом задання використайте TRY...CATCH у транзакціях. Якщо під час виконання операцій UPDATE виникає помилка

(наприклад, через обмеження або невірні дані), транзакція має бути повністю скасована за допомогою `ROLLBACK TRANSACTION`. У блоці `CATCH` виведіть повідомлення про помилку.

Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл `CUSOR.SQL` та оновіть його на своїй сторінці на GitHub.

Завдання 6.

Зробіть транзакцію в якій додається 10'000-100'000 рядків (можна використовувати з попередніх таблиць але потрібно показати, що така таблиця є з багатою кількістю рядків).

Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл `CUSOR.SQL` та оновіть його на своїй сторінці на GitHub.

Завдання 7.

Зробіть транзакцію в якій потрібно модифікувати одну з функцій або процедур додавши на початку та в кінці кожної процедури/функції виведення тимчасових міток для фіксування часу виконання процедури/функції;

Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл `CUSOR.SQL` та оновіть його на своїй сторінці на GitHub.

Завдання 8.

Зробіть таблицю:

Таблиця 1. Час виконання запитів № Опис запуску п/п Час виконання запиту

- Запуск п/п за таблицею (з великою кількістю рядків)
- Запуск з Index таблицею (оптимізований той же самий запит до цієї ж таблиці)

- З використанням Cursor1 п.8 (Виконання того ж самого запиту засобами курсорів)
- З використанням Cursor2 п.9 (Виконання того ж самого запиту засобами курсорів)
- З використанням Cursor2 п.10 (Виконання того ж самого запиту засобами курсорів)

Додайте результат виконання запиту (7) у перший рядок зведеної таблиці. Запит повинен бути «складним» з використанням декількох JOIN, сортування, фільтрування;

Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл CUSOR.SQL та оновіть його на своїй сторінці на GitHub.

Завдання 9.

Додайте результат виконання запиту (7) у другий рядок зведеної таблиці. Обов'язково потрібно додати індекс у відповідності до параметрів запиту. Запит повинен бути «складним» з використанням декількох JOIN, сортування, фільтрування; Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл CUSOR.SQL та оновіть його на своїй сторінці на GitHub.

Завдання 10.

Додайте результат виконання запиту (7) у третій рядок зведеної таблиці. Але запит потрібно реалізувати засобами курсорів T-SQL. Результати виконання повинні бути однакові з попередніми. Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл CUSOR.SQL та оновіть його на своїй сторінці на GitHub.

Завдання 11.

Модифікуйте попередній запит: додавши виконання запиту але в иконайте код двічі між запитами не потрібно висвободжати (DEALLOCATE) курсор, та додайте результат виконання цього запиту у четвертий та п'ятий рядок зведеної таблиці. Результати виконання повинні бути однакові з попередніми. Додайте до звіту.

- Скріншоти виконання запитів та їхніх результатів;
- Зберегти код запитів додавши у файл CUSOR.SQL та оновіть його на своїй сторінці на GitHub.

Завдання 12.

Порівняйте час виконання запитів у зведеній таблиці. Проаналізуйте результати. Яким чином можна прискорити виконання запитів? Який тип запитів може прискорити цей час? В яких випадках потрібно використовувати курсори?

Завдання 13.

Завантажити файли до Git-репозиторію та додати посилання у звіт. Репозиторій повинен бути закритим, а доступ відкритий лише для викладачів практики, лабораторних робіт та лектора (k.korobchinskiy@karazin.ua). В репозиторії створити або оновити README.MD файл додавши використання процедур з коротким поясненням зроблених завдань. Використовуючи мову T-SQL зробити скрипти.

- створення таблиць та змінення структури таблиць БД – SETUP.SQL;
- занесення інформації в таблиці БД – INSERT.SQL.
- зміни/оновлення/видалення інформації у таблицях БД – UPDATE.SQL;
- запитів – QUERY.SQL.
- функцій – FUNCTIONS.SQL.
- курсорів – CUSOR.SQL

Всі фали за архівувати та прикипіти до STM.

Завдання 14.

Додати висновки. До оформити звіт та завантажити на STM.Хід роботи:

Завдання 2.

Варіант №15

Ферма

На фермі має бути кілька ділянок землі, їм присвоєні унікальні номери. Кожна ділянка характеризується площею, наявністю або відсутністю зрошення, посіяною в поточному сезоні культурою. Відомо середня урожайність кожної з вирощуваних культур, а також перелік внесених на кожну ділянку в даному сезоні добрив. Для кожної культури відомо: назва, урожайність по ділянках, чи потрібна їй зрошення, які потрібні добрива. Кожна ділянка обслуговується однією бригадою, але будь-яка бригада може обслуговувати більше однієї ділянки. Бригада має унікальний номер і характеризується: Ф.І.О. бригадира, Ф.І.О. працівників, кількістю одиниць кожного виду сільськогосподарської техніки. Про кожного працівника відомо, якими видами сільськогосподарських машин (однією, кількома, ні однією) він може керувати. Працівники працюють на будь-якій з ділянок бригади, користуючись технікою, яка є в бригаді.

Запити

- ✓ Визначити площу, яка відведена на фермі під кожну культуру з вказівкою, на якій кількості ділянок посіяна кожна культура;
- ✓ Вивести всіх працівників заданої бригади з переліком сільськогосподарської техніки, на якій вони можуть працювати, якщо така техніка в бригаді є, а також визначити те види техніки, на якій можуть працювати члени бригади, але в бригаді такої техніки немає, хоча вона є в інших бригадах
- для ділянок даної бригади знайти культуру з максимальною урожайністю і перевірити, посіяна чи ця культура в поточному сезоні на тому ділянці, де її урожайність максимальна
- ✓ Знайти бригади, які мають заданим видом техніки в кількості, більшій за середню кількість цієї техніки по бригадам на всій фермі;
- ✓ Скласти список ділянок (з іменами їх бригадирів), на яких внесені добрива, не відповідні культурі, або не виконуються вимоги по зрошенню культури.

Транзакції

- ✓ Купити нову сільськогосподарську техніку
- ✓ Найняти/увільнити сільськогосподарських (в тому числі і бригадирів).

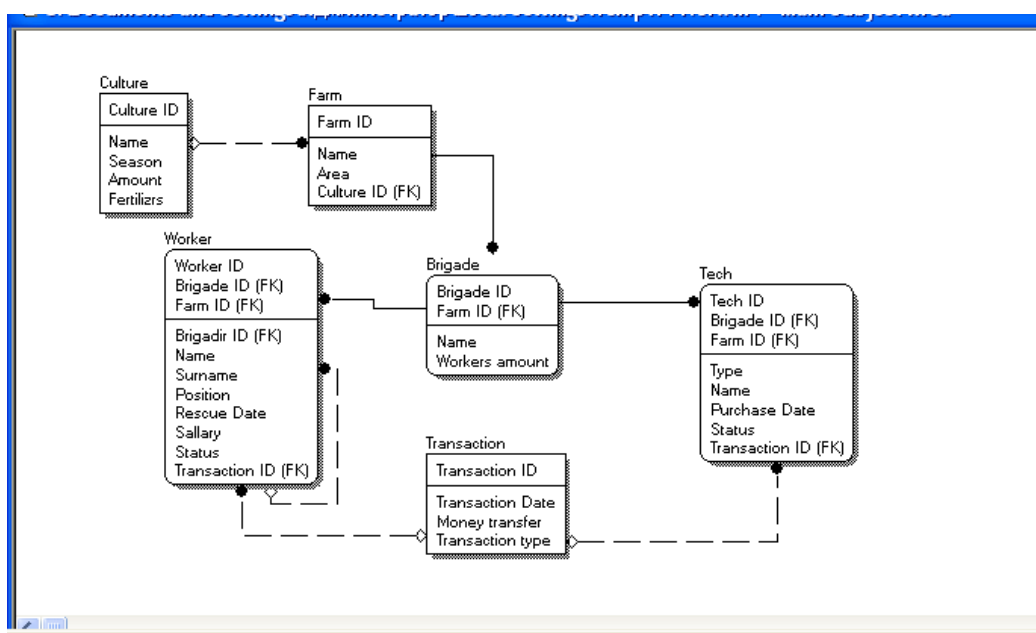


Рисунок 1 - Логічна модель

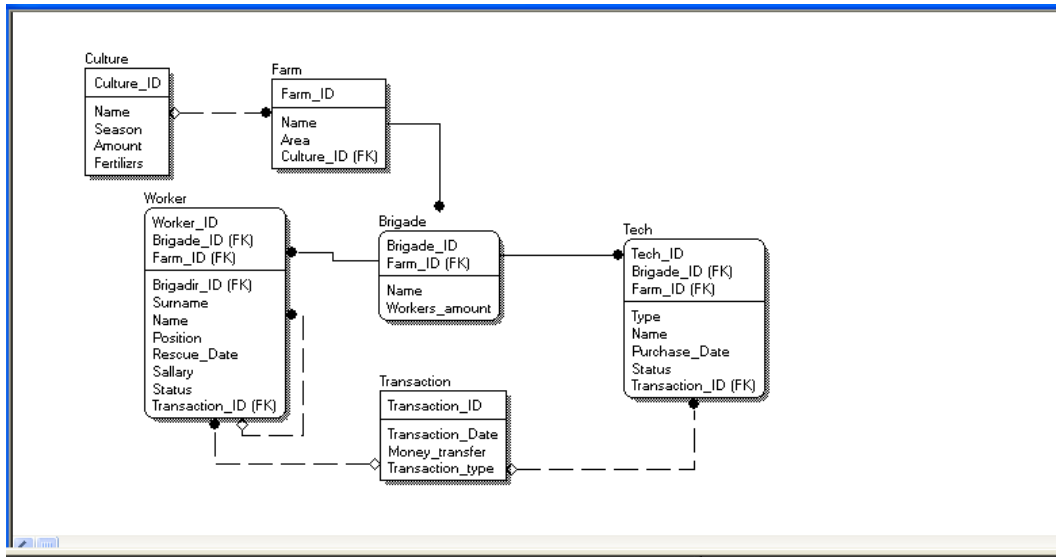


Рисунок 2 - Фізична модель

Завдання 3.

Робимо по прикладу з set і where:

```
BEGIN TRANSACTION;

UPDATE Worker
SET Salary = Salary + 1000
WHERE Worker_ID = 5;

IF (SELECT COUNT(*) FROM Farm WHERE Area > 1000) = 0
    ROLLBACK;
ELSE
    COMMIT;
```

	Worker_ID	Salary		Worker_ID	Salary
1	1	12542.85	1	1	12542.85
2	2	7488.66	2	2	7488.66
3	3	5416.00	3	3	5416.00
4	4	13896.34	4	4	13896.34
5	5	17069.79	5	5	17069.79
6	6	13497.87	6	6	13497.87

Messages

(1 row affected)

Рисунок 3 - до, під час, і після виконання транзакції з ролбек

Як бачимо, рядок «Змінився», але насправді в таблиці ніяких змін немає – тому що в нас нема ферм з площею більше за 1000 і відповідно відбувається відкат.

Завдання 4.

Аналогічно з прикладом:

```

BEGIN TRANSACTION;

UPDATE Worker
SET Salary = Salary + 500
WHERE Worker_ID = -1;

IF @@ERROR <> 0
    ROLLBACK;
ELSE
    COMMIT;

```

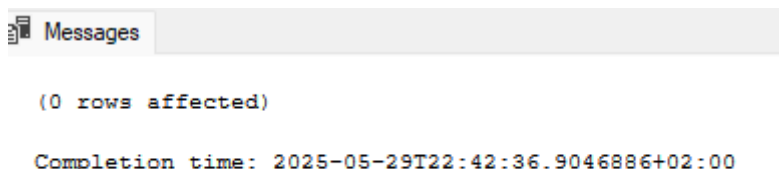


Рисунок 4 - вивід при помилковому запиті

Тут нема що показати, оскільки воркерів з мінусовим айді не існує, то запрос дає помилку, а коли помилка то відкат. Хоча! Насправді, є враження що це не еррор все так круто оброблює, а насправді where не знаходить Worker_ID = -1 і далі комміт. Я чесно намагався спіймати помилку різними способами, і давати чар заміт інт, і коректувати неіснуючи атрибути, і давати повторний первинний ключ – завжди фатальна помилка, і до IF справа просто не доходить. Тому я залишив початкову версію – вона написана по прикладу, і, за логікою, виконує ту ж саму функцію. А насправді помилку зловимо в наступному завданні з CATCH.

Завдання 5.

```

BEGIN TRANSACTION;
BEGIN TRY
    UPDATE Worker
    SET Salary = 'wrong'
    WHERE Worker_ID = 5;

    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;

```



```
PRINT ERROR_MESSAGE();
END CATCH;
```

```
(0 rows affected)
Error converting data type varchar to numeric.

Completion time: 2025-05-29T22:57:50.7519715+02:00
```

Рисунок 5 - скріншот де ми CATCH помилку

Тут все вийшло як треба – вивелося повідомлення про помилку, але не фатальне, інкаше б воно було червоне.

Завдання 6.

Беремо за основу скрипт, який використовували в минулій роботі при додаванні 10к робітників. Робимо з нього транзакцію:

```
BEGIN TRANSACTION;
BEGIN TRY
    DECLARE @CurrentMax INT = (SELECT MAX(Worker_ID) FROM Worker);
    DECLARE @StartID INT =
        CASE
            WHEN @CurrentMax IS NULL THEN 100000
            WHEN @CurrentMax < 100000 THEN 100000
            ELSE @CurrentMax + 1
        END;

    DECLARE @RowCount INT = 10000 + ABS(CHECKSUM(NEWID())) % 90001;

    SELECT DISTINCT Position INTO #Positions FROM Worker WHERE Position
IS NOT NULL;
    SELECT DISTINCT Status INTO #Statuses FROM Worker WHERE Status IS
NOT NULL;
    SELECT DISTINCT Tech_Type INTO #Types FROM Worker WHERE Tech_Type
IS NOT NULL;

    ;WITH E1(N) AS (SELECT 1 UNION ALL SELECT 1),
        E2(N) AS (SELECT 1 FROM E1 a, E1 b), -- 4
        E3(N) AS (SELECT 1 FROM E2 a, E2 b), -- 16
        E4(N) AS (SELECT 1 FROM E3 a, E3 b), -- 256
        E5(N) AS (SELECT 1 FROM E4 a, E4 b), -- 65,536
        E6(N) AS (SELECT 1 FROM E5 a, E2 b), -- 262,144
        E7(N) AS (SELECT 1 FROM E6 a, E2 b), -- 1,048,576
        Numbers AS (
```

```

        SELECT TOP (@RowCount) ROW_NUMBER() OVER (ORDER BY (SELECT
NULL)) AS N
        FROM E7
    ),
    Brigades AS (
        SELECT Brigade_ID, Farm_ID FROM Brigade
    ),
    IndexedBrigades AS (
        SELECT B.*, ROW_NUMBER() OVER (ORDER BY NEWID()) AS rn
        FROM Brigades B
    )

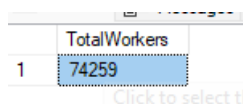
INSERT INTO Worker (
    Worker_ID, Brigade_ID, Farm_ID, Brigadir_ID,
    Name, Surname, Position, Rescue_Date, Salary,
    Status, Transaction_ID, Tech_Type
)
SELECT
    @StartID + N.N,
    B.Brigade_ID,
    B.Farm_ID,
    B.Brigade_ID,
    CONCAT('Name', @StartID + N.N),
    CONCAT('Surname', @StartID + N.N),
    (SELECT TOP 1 Position FROM #Positions ORDER BY NEWID()),
    DATEADD(DAY, -ABS(CHECKSUM(NEWID())) % 1800, GETDATE()),
    5000 + ABS(CHECKSUM(NEWID())) % 15000,
    (SELECT TOP 1 Status FROM #Statuses ORDER BY NEWID()),
    NULL,
    (SELECT TOP 1 Tech_Type FROM #Types ORDER BY NEWID())
FROM Numbers N
CROSS APPLY (
    SELECT TOP 1 Brigade_ID, Farm_ID
    FROM IndexedBrigades
    WHERE rn = ((N.N - 1) % (SELECT COUNT(*) FROM Brigades)) + 1
) B;

DROP TABLE #Positions;
DROP TABLE #Statuses;
DROP TABLE #Types;

COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT ERROR_MESSAGE();
END CATCH;

```

Виконаємо SELECT COUNT(*) AS TotalWorkers FROM Worker;



	TotalWorkers
1	74259

Рисунок 6 - кількість людей після транзакції

Завдання 7.

Оновимо процедуру додавання воркера з PROCEDURES.SQL, трошки ускладнемо його скоалдним селектом з джоїнт, where, order by:

```

ALTER PROCEDURE AddNewWorker
    @Worker_ID INT,
    @Name NVARCHAR(50),
    @Surname NVARCHAR(50),
    @Farm_ID INT,
    @Brigade_ID INT
AS
BEGIN
    DECLARE @StartTime DATETIME2 = SYSDATETIME();

    PRINT 'Start Time: ' + CONVERT(NVARCHAR, @StartTime);

    BEGIN TRANSACTION;

    BEGIN TRY
        IF NOT EXISTS (SELECT 1 FROM Farm WHERE Farm_ID = @Farm_ID)
        BEGIN
            RAISERROR('Farm not found.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        IF NOT EXISTS (
            SELECT 1 FROM Brigade
            WHERE Brigade_ID = @Brigade_ID AND Farm_ID = @Farm_ID
        )
        BEGIN
            RAISERROR('Brigade not found.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        INSERT INTO Worker(Worker_ID, Name, Surname, Farm_ID, Brigade_ID)
        VALUES (@Worker_ID, @Name, @Surname, @Farm_ID, @Brigade_ID);

        COMMIT TRANSACTION;

        SELECT TOP 10
            W.Worker_ID,
            W.Name,
            W.Surname,
            B.Brigade_ID,
            B.Farm_ID,
            F.Name AS FarmName,
            W.Salary,
            W.Rescue_Date
        FROM Worker W
        INNER JOIN Brigade B ON W.Brigade_ID = B.Brigade_ID
    
```

```

INNER JOIN Farm F ON W.Farm_ID = F.Farm_ID
WHERE W.Salary > 8000 AND W.Position IS NOT NULL
ORDER BY W.Rescue_Date DESC;

END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH

DECLARE @EndTime DATETIME2 = SYSDATETIME();
PRINT 'End Time: ' + CONVERT(NVARCHAR, @EndTime);
PRINT 'Execution Duration (ms): ' + CONVERT(NVARCHAR,
DATEDIFF(MILLISECOND, @StartTime, @EndTime));
END;

```

Тепер, після такого запиту, має вивестись час:

```

EXEC AddNewWorker
    @Worker_ID = 200003,
    @Name = 'Test',
    @Surname = 'Worker',
    @Farm_ID = 1,
    @Brigade_ID = 1;

Start Time: 2025-05-30 00:01:15.6258127

(1 row affected)

(10 rows affected)
End Time: 2025-05-30 00:01:15.6734027
Execution Duration (ms): 48

```

Рисунок 7 - час виконання вивівся

Завдання 8.

№	Опис запуску п/п	Час виконання запиту
1	Запуск п/п за таблицею (з великою кількістю рядків)	48 ms
2	Запуск з Index таблицею (оптимізований той же запит)	43 ms
3	З використанням Cursor1 п.8	81 ms
4	З використанням Cursor2 п.9	73 ms
5	З використанням Cursor2 п.10	82 ms

Завдання 9.

```
CREATE NONCLUSTERED INDEX IX_Worker_Salary_Position_RescueDate
ON Worker (Salary, Position, Rescue_Date DESC)
INCLUDE (Worker_ID, Name, Surname, Brigade_ID, Farm_ID);
```

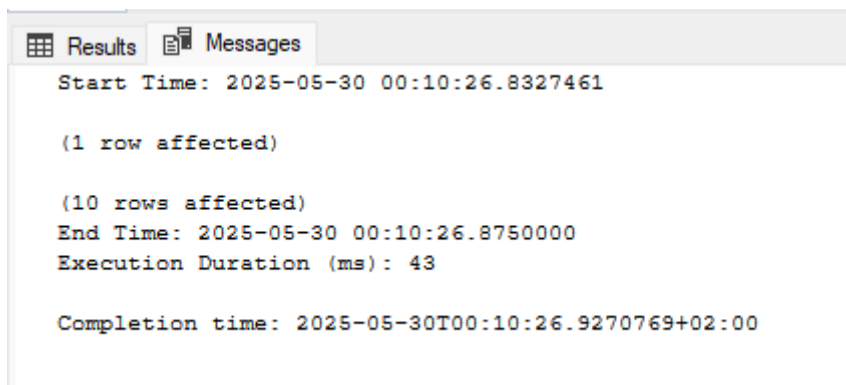


Рисунок 8 - час з індексом

Завдання 10.

Змінюємо під курсор:

```
ALTER PROCEDURE AddNewWorker
    @Worker_ID INT,
    @Name NVARCHAR(50),
    @Surname NVARCHAR(50),
    @Farm_ID INT,
    @Brigade_ID INT
AS
BEGIN
    DECLARE @StartTime DATETIME2 = SYSDATETIME();
    PRINT 'Start Time: ' + CONVERT(NVARCHAR, @StartTime);

    BEGIN TRANSACTION;

    BEGIN TRY
        IF NOT EXISTS (SELECT 1 FROM Farm WHERE Farm_ID = @Farm_ID)
        BEGIN
            RAISERROR('Farm not found.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        IF NOT EXISTS (
            SELECT 1 FROM Brigade
            WHERE Brigade_ID = @Brigade_ID AND Farm_ID = @Farm_ID
        )
        BEGIN
            RAISERROR('Brigade not found.', 16, 1);
            ROLLBACK TRANSACTION;
        END
    END TRY
    CATCH
    BEGIN
        ROLLBACK TRANSACTION;
    END
END
```

```

    RETURN;
END

INSERT INTO Worker(Worker_ID, Name, Surname, Farm_ID, Brigade_ID)
VALUES (@Worker_ID, @Name, @Surname, @Farm_ID, @Brigade_ID);

COMMIT TRANSACTION;

DECLARE @WID INT,
        @WName NVARCHAR(50),
        @WSurname NVARCHAR(50),
        @BID INT,
        @FID INT,
        @FName NVARCHAR(100),
        @Salary MONEY,
        @RescueDate DATE;

DECLARE WorkerCursor CURSOR FOR
    SELECT TOP 10
        W.Worker_ID,
        W.Name,
        W.Surname,
        B.Brigade_ID,
        B.Farm_ID,
        F.Name,
        W.Salary,
        W.Rescue_Date
    FROM Worker W
    INNER JOIN Brigade B ON W.Brigade_ID = B.Brigade_ID
    INNER JOIN Farm F ON W.Farm_ID = F.Farm_ID
    WHERE W.Salary > 8000 AND W.Position IS NOT NULL
    ORDER BY W.Rescue_Date DESC;

OPEN WorkerCursor;
FETCH NEXT FROM WorkerCursor INTO @WID, @WName, @WSurname, @BID,
@FID, @FName, @Salary, @RescueDate;

WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT 'Worker_ID: ' + CAST(@WID AS NVARCHAR) +
        ', Name: ' + @WName +
        ', Surname: ' + @WSurname +
        ', Brigade_ID: ' + CAST(@BID AS NVARCHAR) +
        ', Farm_ID: ' + CAST(@FID AS NVARCHAR) +
        ', FarmName: ' + @FName +
        ', Salary: ' + CAST(@Salary AS NVARCHAR) +
        ', Rescue_Date: ' + CONVERT(NVARCHAR, @RescueDate);

    FETCH NEXT FROM WorkerCursor INTO @WID, @WName, @WSurname,
@BID, @FID, @FName, @Salary, @RescueDate;
END

CLOSE WorkerCursor;

```

```

        DEALLOCATE WorkerCursor;

    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        PRINT 'Error: ' + ERROR_MESSAGE();
    END CATCH

    DECLARE @EndTime DATETIME2 = SYSDATETIME();
    PRINT 'End Time: ' + CONVERT(NVARCHAR, @EndTime);
    PRINT 'Execution Duration (ms): ' + CONVERT(NVARCHAR,
DATEDIFF(MILLISECOND, @StartTime, @EndTime));
END;

```

Start Time: 2025-05-30 00:18:20.0399777

(1 row affected)

```

Worker_ID: 100026, Name: Name100026, Surname: Surname100026, Brigade_ID: 61, Farm_ID: 61, FarmName: Farm 61, Salary: 9327.00, Rescue_Date: 2025-05-29
Worker_ID: 100089, Name: Name100089, Surname: Surname100089, Brigade_ID: 17, Farm_ID: 17, FarmName: Farm 17, Salary: 8302.00, Rescue_Date: 2025-05-29
Worker_ID: 101407, Name: Name101407, Surname: Surname101407, Brigade_ID: 31, Farm_ID: 31, FarmName: Farm 31, Salary: 9842.00, Rescue_Date: 2025-05-29
Worker_ID: 107345, Name: Name107345, Surname: Surname107345, Brigade_ID: 4, Farm_ID: 4, FarmName: Farm 4, Salary: 10887.00, Rescue_Date: 2025-05-29
Worker_ID: 108646, Name: Name108646, Surname: Surname108646, Brigade_ID: 89, Farm_ID: 89, FarmName: Farm 89, Salary: 10597.00, Rescue_Date: 2025-05-29
Worker_ID: 110062, Name: Name110062, Surname: Surname110062, Brigade_ID: 46, Farm_ID: 46, FarmName: Farm 46, Salary: 12231.00, Rescue_Date: 2025-05-29
Worker_ID: 113614, Name: Name113614, Surname: Surname113614, Brigade_ID: 72, Farm_ID: 72, FarmName: Farm 72, Salary: 12624.00, Rescue_Date: 2025-05-29
Worker_ID: 117730, Name: Name117730, Surname: Surname117730, Brigade_ID: 40, Farm_ID: 40, FarmName: Farm 40, Salary: 15133.00, Rescue_Date: 2025-05-29
Worker_ID: 122932, Name: Name122932, Surname: Surname122932, Brigade_ID: 25, Farm_ID: 25, FarmName: Farm 25, Salary: 17278.00, Rescue_Date: 2025-05-29
Worker_ID: 123636, Name: Name123636, Surname: Surname123636, Brigade_ID: 53, Farm_ID: 53, FarmName: Farm 53, Salary: 19160.00, Rescue_Date: 2025-05-29
End Time: 2025-05-30 00:18:20.1208915
Execution Duration (ms): 81

```

Рисунок 9 - вивід через курсори

Завдання 11.

```

ALTER PROCEDURE AddNewWorker
    @Worker_ID INT,
    @Name NVARCHAR(50),
    @Surname NVARCHAR(50),
    @Farm_ID INT,
    @Brigade_ID INT
AS
BEGIN
    DECLARE @StartTime DATETIME2 = SYSDATETIME();
    PRINT 'Start Time (вся процедура): ' + CONVERT(NVARCHAR, @StartTime);

    BEGIN TRANSACTION;

    BEGIN TRY
        IF NOT EXISTS (SELECT 1 FROM Farm WHERE Farm_ID = @Farm_ID)
        BEGIN
            RAISERROR('Farm not found.', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        IF NOT EXISTS (
            SELECT 1 FROM Brigade
            WHERE Brigade_ID = @Brigade_ID AND Farm_ID = @Farm_ID

```

```

)
BEGIN
    RAISERROR('Brigade not found.', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END

INSERT INTO Worker(Worker_ID, Name, Surname, Farm_ID, Brigade_ID)
VALUES (@Worker_ID, @Name, @Surname, @Farm_ID, @Brigade_ID);

COMMIT TRANSACTION;

DECLARE @WID INT,
        @WName NVARCHAR(50),
        @WSurname NVARCHAR(50),
        @BID INT,
        @FID INT,
        @FName NVARCHAR(100),
        @Salary MONEY,
        @RescueDate DATE;

IF CURSOR_STATUS('global', 'WorkerCursor') >= -1
    CLOSE WorkerCursor;

DECLARE WorkerCursor CURSOR GLOBAL FOR
    SELECT TOP 10
        W.Worker_ID,
        W.Name,
        W.Surname,
        B.Brigade_ID,
        B.Farm_ID,
        F.Name,
        W.Salary,
        W.Rescue_Date
    FROM Worker W
    INNER JOIN Brigade B ON W.Brigade_ID = B.Brigade_ID
    INNER JOIN Farm F ON W.Farm_ID = F.Farm_ID
    WHERE W.Salary > 8000 AND W.Position IS NOT NULL
    ORDER BY W.Rescue_Date DESC;

--1 cursor
DECLARE @Start1 DATETIME2 = SYSDATETIME();

OPEN WorkerCursor;
FETCH NEXT FROM WorkerCursor INTO @WID, @WName, @WSurname, @BID,
@FID, @FName, @Salary, @RescueDate;

PRINT 'Cursor Pass 1';
WHILE @@FETCH_STATUS = 0

```



```

BEGIN
    PRINT 'Worker_ID: ' + CAST(@WID AS NVARCHAR) +
        ', Name: ' + @WName +
        ', Surname: ' + @WSurname +
        ', Brigade_ID: ' + CAST(@BID AS NVARCHAR) +
        ', Farm_ID: ' + CAST(@FID AS NVARCHAR) +
        ', FarmName: ' + @FName +
        ', Salary: ' + CAST(@Salary AS NVARCHAR) +
        ', Rescue_Date: ' + CONVERT(NVARCHAR, @RescueDate);

    FETCH NEXT FROM WorkerCursor INTO @WID, @WName, @WSurname,
@BID, @FID, @FName, @Salary, @RescueDate;
END
DECLARE @End1 DATETIME2 = SYSDATETIME();
PRINT 'Pass 1 Duration (ms): ' + CAST(DATEDIFF(MILLISECOND,
@Start1, @End1) AS NVARCHAR);
CLOSE WorkerCursor;

---2 cursor
DECLARE @Start2 DATETIME2 = SYSDATETIME();

OPEN WorkerCursor;
FETCH NEXT FROM WorkerCursor INTO @WID, @WName, @WSurname, @BID,
@FID, @FName, @Salary, @RescueDate;

PRINT 'Cursor Pass 2';
WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT 'Worker_ID: ' + CAST(@WID AS NVARCHAR) +
        ', Name: ' + @WName +
        ', Surname: ' + @WSurname +
        ', Brigade_ID: ' + CAST(@BID AS NVARCHAR) +
        ', Farm_ID: ' + CAST(@FID AS NVARCHAR) +
        ', FarmName: ' + @FName +
        ', Salary: ' + CAST(@Salary AS NVARCHAR) +
        ', Rescue_Date: ' + CONVERT(NVARCHAR, @RescueDate);

    FETCH NEXT FROM WorkerCursor INTO @WID, @WName, @WSurname,
@BID, @FID, @FName, @Salary, @RescueDate;
END
DECLARE @End2 DATETIME2 = SYSDATETIME();
PRINT 'Pass 2 Duration (ms): ' + CAST(DATEDIFF(MILLISECOND,
@Start2, @End2) AS NVARCHAR);
CLOSE WorkerCursor;

DEALLOCATE WorkerCursor;

END TRY
BEGIN CATCH

```

```

ROLLBACK TRANSACTION;

PRINT 'Error: ' + ERROR_MESSAGE();

END CATCH

DECLARE @EndTime DATETIME2 = SYSDATETIME();
PRINT 'End Time for all: ' + CONVERT(NVARCHAR, @EndTime);
PRINT 'Execution Duration Total (ms): ' + CONVERT(NVARCHAR,
DATEDIFF(MILLISECOND, @StartTime, @EndTime));
END;

```

Start Time (??? ?????????): 2025-05-30 00:35:26.0970986

(1 row affected)

Cursor Pass 1

Worker_ID: 100026, Name: Name100026, Surname: Surname100026, Brigade_ID: 61, Farm_ID: 61, FarmName: Farm 61, Salary: 9327.00, Rescue_Date: 2025-05-29
Worker_ID: 100089, Name: Name100089, Surname: Surname100089, Brigade_ID: 17, Farm_ID: 17, FarmName: Farm 17, Salary: 8302.00, Rescue_Date: 2025-05-29
Worker_ID: 101407, Name: Name101407, Surname: Surname101407, Brigade_ID: 31, Farm_ID: 31, FarmName: Farm 31, Salary: 9842.00, Rescue_Date: 2025-05-29
Worker_ID: 107345, Name: Name107345, Surname: Surname107345, Brigade_ID: 4, Farm_ID: 4, FarmName: Farm 4, Salary: 10887.00, Rescue_Date: 2025-05-29
Worker_ID: 108646, Name: Name108646, Surname: Surname108646, Brigade_ID: 89, Farm_ID: 89, FarmName: Farm 89, Salary: 10597.00, Rescue_Date: 2025-05-29
Worker_ID: 110062, Name: Name110062, Surname: Surname110062, Brigade_ID: 46, Farm_ID: 46, FarmName: Farm 46, Salary: 12231.00, Rescue_Date: 2025-05-29
Worker_ID: 113614, Name: Name113614, Surname: Surname113614, Brigade_ID: 72, Farm_ID: 72, FarmName: Farm 72, Salary: 12624.00, Rescue_Date: 2025-05-29
Worker_ID: 117730, Name: Name117730, Surname: Surname117730, Brigade_ID: 40, Farm_ID: 40, FarmName: Farm 40, Salary: 15133.00, Rescue_Date: 2025-05-29
Worker_ID: 122932, Name: Name122932, Surname: Surname122932, Brigade_ID: 25, Farm_ID: 25, FarmName: Farm 25, Salary: 17278.00, Rescue_Date: 2025-05-29
Worker_ID: 123636, Name: Name123636, Surname: Surname123636, Brigade_ID: 53, Farm_ID: 53, FarmName: Farm 53, Salary: 19160.00, Rescue_Date: 2025-05-29

Pass 1 Duration (ms): 73

Cursor Pass 2

Worker_ID: 100026, Name: Name100026, Surname: Surname100026, Brigade_ID: 61, Farm_ID: 61, FarmName: Farm 61, Salary: 9327.00, Rescue_Date: 2025-05-29
Worker_ID: 100089, Name: Name100089, Surname: Surname100089, Brigade_ID: 17, Farm_ID: 17, FarmName: Farm 17, Salary: 8302.00, Rescue_Date: 2025-05-29
Worker_ID: 101407, Name: Name101407, Surname: Surname101407, Brigade_ID: 31, Farm_ID: 31, FarmName: Farm 31, Salary: 9842.00, Rescue_Date: 2025-05-29
Worker_ID: 107345, Name: Name107345, Surname: Surname107345, Brigade_ID: 4, Farm_ID: 4, FarmName: Farm 4, Salary: 10887.00, Rescue_Date: 2025-05-29
Worker_ID: 108646, Name: Name108646, Surname: Surname108646, Brigade_ID: 89, Farm_ID: 89, FarmName: Farm 89, Salary: 10597.00, Rescue_Date: 2025-05-29
Worker_ID: 110062, Name: Name110062, Surname: Surname110062, Brigade_ID: 46, Farm_ID: 46, FarmName: Farm 46, Salary: 12231.00, Rescue_Date: 2025-05-29
Worker_ID: 113614, Name: Name113614, Surname: Surname113614, Brigade_ID: 72, Farm_ID: 72, FarmName: Farm 72, Salary: 12624.00, Rescue_Date: 2025-05-29
Worker_ID: 117730, Name: Name117730, Surname: Surname117730, Brigade_ID: 40, Farm_ID: 40, FarmName: Farm 40, Salary: 15133.00, Rescue_Date: 2025-05-29
Worker_ID: 122932, Name: Name122932, Surname: Surname122932, Brigade_ID: 25, Farm_ID: 25, FarmName: Farm 25, Salary: 17278.00, Rescue_Date: 2025-05-29
Worker_ID: 123636, Name: Name123636, Surname: Surname123636, Brigade_ID: 53, Farm_ID: 53, FarmName: Farm 53, Salary: 19160.00, Rescue_Date: 2025-05-29

Pass 2 Duration (ms): 82

End Time for all: 2025-05-30 00:35:26.2564189

Execution Duration Total (ms): 159

Completion time: 2025-05-30T00:35:26.2694614+02:00

Рисунок 10 - два курсори без deallocate

Завдання 12.

Час виконання звичайного запиту без індексу склав 48 мс, з індексом — 43 мс, що демонструє незначне, але стабільне покращення продуктивності за рахунок оптимізації. Використання курсора значно збільшує час — від 73 до 82 мс, оскільки обробка кожного рядка відбувається поетапно, що створює додаткові витрати ресурсів. Це підтверджує, що курсори менш ефективні для великих наборів даних. Прискорити запити можна за рахунок створення індексів на полях, які беруть участь у фільтрації та сортуванні. Курсори доцільно використовувати лише в тих випадках, коли потрібна покрокова логіка або операції, які неможливо реалізувати через стандартні SQL-конструкції. У загальному випадку оптимізовані селекції дають кращу продуктивність.

Висновок:

У ході лабораторної роботи було створено та протестовано збережену процедуру з використанням курсорів, що дозволило оцінити ефективність різних підходів до обробки даних. Було зафіксовано час виконання стандартного запиту, оптимізованого запиту з індексами, а також курсорних реалізацій. Аналіз показав, що індексовані запити працюють найшвидше, тоді як курсори споживають більше ресурсів і мають більший час виконання. Це свідчить про доцільність використання курсорів лише в тих випадках, коли потрібна покрокова обробка даних. Отже, для підвищення продуктивності доцільно застосовувати індексацію та уникати курсорів, якщо це можливо.

GitHub з файлами практичної: <https://github.com/ddanq4/3-obd/tree/main/2%20semestr/lab6>